

LANGCHAIN İLE BELLEK (MEMORY) YÖNETİMİ

ÇALIŞMA RAPORU

LLM Uygulamalarında Statelessness (Durumsuzluk) Sorununa Çözümler, Kod Örnekleri ve Analogiler

1. Giriş ve LLM'lerde Bellek İhtiyacı

Büyük Dil Modelleri (LLM'ler) doğaları gereği **durumsuz (stateless)** yapılardır. Sisteme gönderilen her bir API çağırısı, geçmişteki işlemlerden tamamen bağımsız ve habersiz olarak ele alınır. Ancak, bir sohbet robotu (chatbot) veya akıllı asistan tasarlarırken, kullanıcıyla kesintisiz ve anlamlı bir diyalog akışı sürdürmek zorunludur. LangChain gibi modern geliştirme çerçeveleri, dil modellerine yapay bir bellek kazandırmak için gelişmiş bellek yönetim modülleri sunar.

Bellek Neden Zorunludur?

Temel Çalışma Prensipleri: Chatbotların geçmişi hatırlama yeteneği, dil modelinin kendisinden değil, etrafındaki kod katmanından gelir. Bu katman, her yeni turda o ana kadar gerçekleşmiş olan tüm konuşma geçmişini otomatik olarak modele girdi (context) olarak iletir. Ancak bu yöntem konuşma uzadıkça **Maliyet (Token Artışı)** ve **Bağlam Sınırı (Context Window)** gibi iki büyük teknik kısıta yol açar. Bu kısıtları yönetmek için doğru bellek tipini seçmek kritik önem taşır.

2. LangChain Bellek Tipleri ve Detaylı Analizleri

2.1. ConversationBufferMemory (Tam Sohbet Belleği)

Bu bellek tipi, konuşmadaki tüm girdileri ve çıktıları ham haliyle bir tampon bellekte (buffer) saklar. Her yeni tura girerken bu tamponun tamamı modele gönderilir.

Günlük Hayat Analogisi: Konuşulan her kelimeyi harfi harfine bir deftere not eden ve her yeni sorunuza cevap vermeden önce tüm defteri baştan sona okuyan aşırı titiz bir asistan. Hafızası kusursuzdur ancak defter kalınlaştıkça asistanın okuması çok uzun sürer ve her okuma için ödediğiniz maliyet katlanarak artar.

```

from langchain.chat_models import ChatOpenAI
from langchain.chains import ConversationChain
from langchain.memory import ConversationBufferMemory

# LLM ve Tam Bellek Yapılandırması
llm = ChatOpenAI(temperature=0.0)
memory = ConversationBufferMemory()
conversation = ConversationChain(llm=llm, memory=memory, verbose=True)

# 1. Tur: Kullanıcı adını söyler
conversation.predict(input="Merhaba, ben Andrew")
# Çıktı: Hello Andrew, it's nice to meet you.

# 2. Tur: İşlem sorulur
conversation.predict(input="1 artı 1 kaç yapar?")
# Çıktı: 1 plus 1 is 2.

# 3. Tur: Geçmiş bilgisi sınıanır
conversation.predict(input="Benim adım neydi?")
# Çıktı: Your name is Andrew, as you mentioned earlier.

```

Sistem Çalışma Biçimi: `verbose=True` yapıldığında, üçüncü turda LangChain'in dil modeline gönderdiği gerçek şablon (prompt) şu şekildedir:

"The following is a friendly conversation between a human and an AI..."

Human: Merhaba, ben Andrew

AI: Hello Andrew, it's nice to meet you.

Human: 1 artı 1 kaç yapar?

AI: 1 plus 1 is 2

Human: Benim adım neydi?"

Bellek, tüm geçmişini metin tamponunda (`memory.buffer`) saklar ve her seferinde modele yollar.

2.2. ConversationBufferWindowMemory (Pencereli Sohbet Belleği)

Belleğin sınırsızca büyümesini engellemek için sadece son `k` adet konuşma turunu (exchange) saklar. Belirlenen pencere sınırının dışındaki eski turlar tamamen unutulur.

Günlük Hayat Analojisi: Sadece son konuştuğu 1-2 konuyu aklında tutabilen kısa süreli hafızaya sahip bir insan. Örneğin `k=1` ise, sadece bir saniye önce ne konuştuğunuzu hatırlar, daha öncesini tamamen unutmuştur.

```

from langchain.memory import ConversationBufferWindowMemory

# k=1: Sadece en son yapılan 1 sohbet turunu hafızada tutar
memory = ConversationBufferWindowMemory(k=1)
memory.save_context({"input": "Merhaba, ben Andrew"}, {"output": "Merhaba Andrew!"})
memory.save_context({"input": "1 artı 1 kaç yapar?"}, {"output": "1 artı 1 eşittir 2."})

# Bellek içeriğini yükleme
print(memory.load_memory_variables({}))
# Çıktı: {'history': 'Human: 1 artı 1 kaç yapar?\nAI: 1 artı 1 eşittir 2.'}

```

Sistem Çalışma Biçimi: `k=1` ayarlandığı için bellek ilk turu ('Merhaba, ben Andrew') tamamen silmiştir. Bu aşamada modele *"Benim adım neydi?"* diye sorulursa, model *"Maalesef bu bilgiye sahip değilim"* yanıtını verir. Belleğin taşmasını engellemek için idealdir ancak eski kritik bilgilerin kaybolmasına yol açar.

2.3. ConversationTokenBufferMemory (Token Sınırlı Sohbet Belleği)

Konuşma geçmişini tur sayısına göre değil, doğrudan **maksimum token sayısı** sınırına göre saklar. Belirlenen token limiti aşıldığında, en eski mesajlar sırayla bellekten kırılır.

Günlük Hayat Analogisi: Sadece 50 kelimelik yazma alanı olan bir küçük not kağıdı. Kağıt dolduğunda, en alta yeni bir şey yazabilmek için en üstteki en eski satırları silgiyle silen asistan.

```
from langchain.memory import ConversationTokenBufferMemory
from langchain.chat_models import ChatOpenAI

llm = ChatOpenAI(temperature=0.0)
# En fazla 50 token sınırlı bellek oluşturma (LLM parametresi zorunludur)
memory = ConversationTokenBufferMemory(llm=llm, max_token_limit=50)

memory.save_context({"input": "Yapay zeka nedir?"}, {"output": "Harikadır."})
memory.save_context({"input": "Geriye yayılım nedir?"}, {"output": "Beautiful."})
memory.save_context({"input": "Sohbet robotu nedir?"}, {"output": "Charming."})

print(memory.load_memory_variables({}))
```

Kritik Detay: Bu bellek tipinde *llm* parametresinin verilmesi zorunludur. Çünkü her dil modelinin (GPT-4, Claude vb.) metinleri token'lara bölme ve sayma yöntemleri (tokenizer) farklıdır. Bu parametre, doğru token sayımı yapılmasını sağlayarak bütçe ve maliyet kontrolünü doğrudan garanti altına alır.

2.4. ConversationSummaryBufferMemory (Özetlemeli Sohbet Belleği)

En akıllı bellek tiplerinden biridir. Belirlenen token limitinin altındaki son mesajları ham haliyle korurken, limitin üstünde kalan eski mesajları arka planda bir LLM çağırısı yaparak özetler ve bellekte bu özet metni saklar.

Günlük Hayat Analogisi: Konuşma uzadıkça, eski detayları unutmamak adına onları sizin için tek paragraflık bir özet haline getiren, ancak son konuşulan birkaç cümleyi de kelimesi kelimesine aklında tutan çok profesyonel bir asistan.

```
from langchain.memory import ConversationSummaryBufferMemory

# En fazla 100 token ham mesaj sınırlı bellek oluşturulması
memory = ConversationSummaryBufferMemory(llm=llm, max_token_limit=100)

schedule = "Saat 08:00'de ürün ekibiyle toplantı var. Sunum dosyasını unutma. Saat 12:00'de müşteri ile öğle yemeği"
memory.save_context({"input": "Bugün programda ne var?"}, {"output": schedule})
memory.save_context({"input": "Harika, teşekkürler"}, {"output": "Rica ederim, başka bir şey var mı?"})

# Limit aşıldığında geçmiş otomatik olarak özetlenir
print(memory.load_memory_variables({}))
```

Sistem Çalışma Biçimi: Sistem arka planda OpenAI API'sine gizli bir prompt yollayarak eski konuşmaları özetletir. Yüklenen bellek şu şekildedir:

"System: Human ve AI günlük konuşma yaptıktan sonra programı sordu. AI sabah 8:00'deki ürün ekibi toplantısını ve 12:00'deki müşteri yemeğini hatırlattı."

Bu sayede hem token tasarrufu en üst düzeye çıkarılır hem de konuşmadaki bağlamsal bütünlük korunmuş olur.

3. Gelişmiş Bellek Yapıları ve Mimari Öneriler

Büyük ölçekli ve kurumsal LangChain projelerinde, sadece yukarıdaki temel bellekler değil, farklı ihtiyaçlara yönelik tasarlanmış daha hibrit ve kalıcı çözümler kullanılır:

- **VectorStoreRetrieverMemory (Vektör Veritabanı Belleği):** Konuşma geçmişini veya alınan bilgileri embedding (vektör gömme) olarak bir vektör veritabanında saklar. Konuşma sırasında, geçmişin tamamını göndermek yerine, yalnızca o an sorulan soruya anlamsal olarak en yakın olan geçmiş bloklarını bulup modele bağlam olarak sunar.
- **EntityMemory (Varlık Belleği):** Konuşmada geçen belirli kişiler, nesneler veya kurumlar hakkında yapılandırılmış bilgileri (örn: bir arkadaşın hobileri, bir projenin teslim tarihi) hatırlar ve bunları özel bir şekilde saklar.
- **Çoklu Bellek Sistemleri (Hybrid Memory):** Gerçek uygulamalarda birden fazla bellek tipi aynı anda entegre edilebilir. Örneğin, ana konuşma akışı için *ConversationSummaryBufferMemory* kullanılırken, kullanıcıyla ilgili özel detayları ayrıca saklamak için sisteme *EntityMemory* eklenebilir.
- **Kalıcı Depolama Entegrasyonu (Persistent Memory):** RAM üzerinde tutulan bellekler oturum kapandığında yok olur. Üretim ortamındaki sistemlerde tüm sohbet geçmişi SQL, Redis veya MongoDB gibi geleneksel veritabanlarında saklanarak oturumlar arası kalıcılık ve denetlenebilirlik (audit) sağlanır.

4. Bellek Tipleri Karşılaştırma Matrisi

Bellek Tipi	Sınırlandırma Kriteri	Token Maliyeti	En Uygun Kullanım Senaryosu
Conversation BufferMemory	Sınır yok (Tüm geçmiş)	Çok Yüksek (Konuşmayla katlanır)	Kısa ve kritik detayların kaçırılmaması gereken özel diyaloglar.
Conversation BufferWindowMemory	Son k sohbet turu	Düşük ve Sabit	Sadece son turların önemli olduğu, geçmişin kritik olmadığı sohbetler.
Conversation TokenBufferMemory	Maksimum Token Limiti	Düşük ve Kontrollü (Bütçe Dostu)	Doğrudan maliyet yönetimi ve katı API bütçesi olan ticari projeler.
Conversation SummaryBufferMemory	Token Sınırı + LLM Özeti	Orta (Özetleme için ek LLM çağırısı yapılır)	Uzun süren ve geçmiş konuların ana fikrinin kaybolmaması gereken büyük projeler.

Sonuç ve Öneri: LangChain bellek bileşenleri, LLM tabanlı uygulamalar geliştirirken kullanıcı deneyimini doğrudan etkileyen en kritik unsurlardan biridir. Geliştirme yaparken projenin bütçesi, beklenen konuşma uzunluğu ve geçmiş verilerin kritiklik derecesine göre bu bellek yapılarından biri veya birkaçı entegre edilerek optimize edilmiş sohbet sistemleri oluşturulabilir.